

## Assignment 11.3 Ai Assisted Coding

Ht.no: 2303A51983

Batch: 06

### Task 1:

Smart Contact Manager (Arrays & Linked Lists)

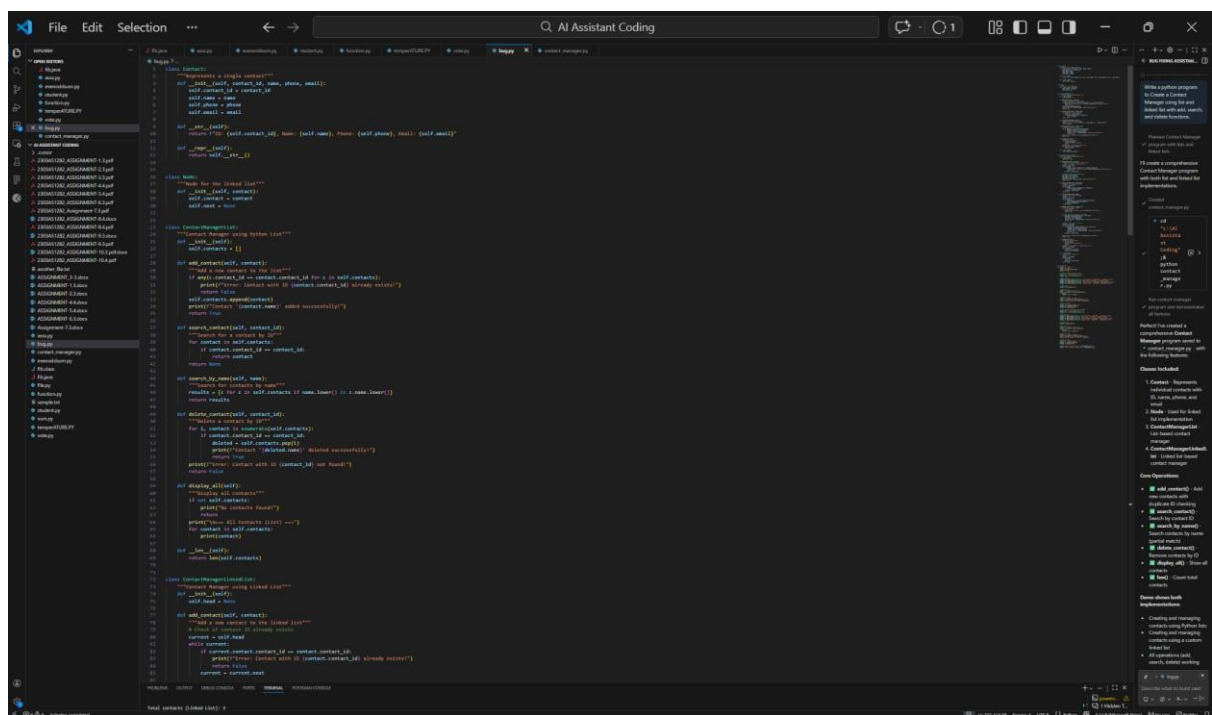
Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

### Prompt:

Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.

### Code:



```
class Contact:
    """Represents a single contact"""
    def __init__(self, contact_id, name, phone, email):
        self.contact_id = contact_id
        self.name = name
        self.phone = phone
        self.email = email

    def __str__(self):
        return f"Contact {self.contact_id}: Name: {self.name}, Phone: {self.phone}, Email: {self.email}"

class ContactManager:
    """Contact Manager using Python List"""
    def __init__(self):
        self.contacts = []

    def add_contact(self, contact):
        """Add a new contact to the list"""
        if not contact_id or not name or not phone or not email:
            print("Error: Contact ID, Name, Phone, and Email are required")
            return False
        self.contacts.append(contact)
        print(f"Contact {contact.contact_id} added successfully")
        return True

    def search_contact(self, contact_id):
        """Search for a contact by ID"""
        for contact in self.contacts:
            if contact.contact_id == contact_id:
                return contact
        return None

    def search_by_name(self, name):
        """Search for a contact by name"""
        results = [c for c in self.contacts if c.name.lower() == name.lower()]
        return results

    def delete_contact(self, contact_id):
        """Delete a contact by ID"""
        for i, contact in enumerate(self.contacts):
            if contact.contact_id == contact_id:
                del self.contacts[i]
                print(f"Contact {contact_id} deleted successfully")
                return True
        print(f"Error: Contact with ID {contact_id} not found")
        return False

    def display_all(self):
        """Display all contacts"""
        if not self.contacts:
            print("No contacts found")
        else:
            print(f"Total contacts: {len(self.contacts)}")
            for contact in self.contacts:
                print(contact)

    def __str__(self):
        return str(self.contacts)

class ContactManagerLinkedList:
    """Contact Manager using Python List"""
    def __init__(self):
        self.head = None

    def add_contact(self, contact):
        """Add a new contact to the linked list"""
        if not contact_id or not name or not phone or not email:
            print("Error: Contact ID, Name, Phone, and Email are required")
            return False
        new_contact = Contact(contact_id, name, phone, email)
        if self.head is None:
            self.head = new_contact
        else:
            current = self.head
            while current.next is not None:
                current = current.next
            current.next = new_contact
        print(f"Contact {contact_id} added successfully")
        return True

    def search_contact(self, contact_id):
        """Search for a contact by ID"""
        current = self.head
        while current is not None:
            if current.contact_id == contact_id:
                return current
            current = current.next
        return None

    def search_by_name(self, name):
        """Search for a contact by name"""
        results = []
        current = self.head
        while current is not None:
            if current.name.lower() == name.lower():
                results.append(current)
            current = current.next
        return results

    def delete_contact(self, contact_id):
        """Delete a contact by ID"""
        if self.head is None:
            print("No contacts found")
            return False
        current = self.head
        previous = None
        while current is not None:
            if current.contact_id == contact_id:
                if previous is None:
                    self.head = current.next
                else:
                    previous.next = current.next
                print(f"Contact {contact_id} deleted successfully")
                return True
            previous = current
            current = current.next
        print(f"Error: Contact with ID {contact_id} not found")
        return False

    def display_all(self):
        """Display all contacts"""
        if self.head is None:
            print("No contacts found")
        else:
            print(f"Total contacts: {len(self.contacts)}")
            current = self.head
            while current is not None:
                print(current)
                current = current.next
```

```
File Edit Selection *** Q AI Assistant Coding
# ContactManager.py
class ContactManager:
    def __init__(self):
        self.contacts = {}

    def add_contact(self, name, phone, email):
        """Add a new contact to the list"""
        if name in self.contacts:
            print(f"Contact '{name}' already exists!")
            return False
        self.contacts[name] = {"phone": phone, "email": email}
        return True

    def search_contact(self, name):
        """Search for a contact by name"""
        if name in self.contacts:
            return self.contacts[name]
        return None

    def search_by_name(self, name):
        """Search for contacts by name"""
        results = []
        for name, contact in self.contacts.items():
            if name.startswith(name.lower()):
                results.append(contact)
        return results

    def delete_contact(self, name):
        """Delete a contact by name"""
        if name in self.contacts:
            del self.contacts[name]
            print(f"Contact '{name}' deleted successfully!")
            return True
        return False

    def update_contact(self, name, phone, email):
        """Update a contact's phone and email"""
        if name in self.contacts:
            self.contacts[name]["phone"] = phone
            self.contacts[name]["email"] = email
            print(f"Contact '{name}' updated successfully!")
            return True
        return False

    def display_contacts(self):
        """Display all contacts"""
        if not self.contacts:
            print("No contacts found!")
            return
        for name, contact in self.contacts.items():
            print(f"Name: {name}, Phone: {contact['phone']}, Email: {contact['email']}")

    def __str__(self):
        return str(self.contacts)

# Main Program
if __name__ == "__main__":
    manager = ContactManager()

    # Add contacts
    manager.add_contact("John Doe", "123-456-7890", "john.doe@example.com")
    manager.add_contact("Jane Smith", "987-654-3210", "jane.smith@example.com")
    manager.add_contact("Bob Johnson", "555-111-2222", "bob.johnson@example.com")

    # Search for a contact
    name = input("Enter name to search: ")
    contact = manager.search_contact(name)
    if contact:
        print(f"Found contact: {contact}")
    else:
        print("Contact not found!")

    # Search by name
    name = input("Enter name to search by: ")
    results = manager.search_by_name(name)
    if results:
        print(f"Search results for '{name}':")
        for result in results:
            print(f"Name: {result['name']}, Phone: {result['phone']}, Email: {result['email']}")
    else:
        print("No results found!")

    # Delete a contact
    name = input("Enter name to delete: ")
    if manager.delete_contact(name):
        print(f"Contact '{name}' deleted successfully!")
    else:
        print("Contact not found!")

    # Update a contact
    name = input("Enter name to update: ")
    phone = input("Enter new phone: ")
    email = input("Enter new email: ")
    if manager.update_contact(name, phone, email):
        print(f"Contact '{name}' updated successfully!")
    else:
        print("Contact not found!")

    # Display all contacts
    manager.display_contacts()

    print("Total contacts: ", len(manager.contacts))
```

```
File Edit Selection *** Q AI Assistant Coding
# ContactManager.py
class ContactManager:
    def __init__(self):
        self.contacts = {}

    def add_contact(self, name, phone, email):
        """Add a new contact to the list"""
        if name in self.contacts:
            print(f"Contact '{name}' already exists!")
            return False
        self.contacts[name] = {"phone": phone, "email": email}
        return True

    def search_contact(self, name):
        """Search for a contact by name"""
        if name in self.contacts:
            return self.contacts[name]
        return None

    def search_by_name(self, name):
        """Search for contacts by name"""
        results = []
        for name, contact in self.contacts.items():
            if name.startswith(name.lower()):
                results.append(contact)
        return results

    def delete_contact(self, name):
        """Delete a contact by name"""
        if name in self.contacts:
            del self.contacts[name]
            print(f"Contact '{name}' deleted successfully!")
            return True
        return False

    def update_contact(self, name, phone, email):
        """Update a contact's phone and email"""
        if name in self.contacts:
            self.contacts[name]["phone"] = phone
            self.contacts[name]["email"] = email
            print(f"Contact '{name}' updated successfully!")
            return True
        return False

    def display_contacts(self):
        """Display all contacts"""
        if not self.contacts:
            print("No contacts found!")
            return
        for name, contact in self.contacts.items():
            print(f"Name: {name}, Phone: {contact['phone']}, Email: {contact['email']}")

    def __str__(self):
        return str(self.contacts)

# Main Program
if __name__ == "__main__":
    manager = ContactManager()

    # Add contacts
    manager.add_contact("John Doe", "123-456-7890", "john.doe@example.com")
    manager.add_contact("Jane Smith", "987-654-3210", "jane.smith@example.com")
    manager.add_contact("Bob Johnson", "555-111-2222", "bob.johnson@example.com")

    # Search for a contact
    name = input("Enter name to search: ")
    contact = manager.search_contact(name)
    if contact:
        print(f"Found contact: {contact}")
    else:
        print("Contact not found!")

    # Search by name
    name = input("Enter name to search by: ")
    results = manager.search_by_name(name)
    if results:
        print(f"Search results for '{name}':")
        for result in results:
            print(f"Name: {result['name']}, Phone: {result['phone']}, Email: {result['email']}")
    else:
        print("No results found!")

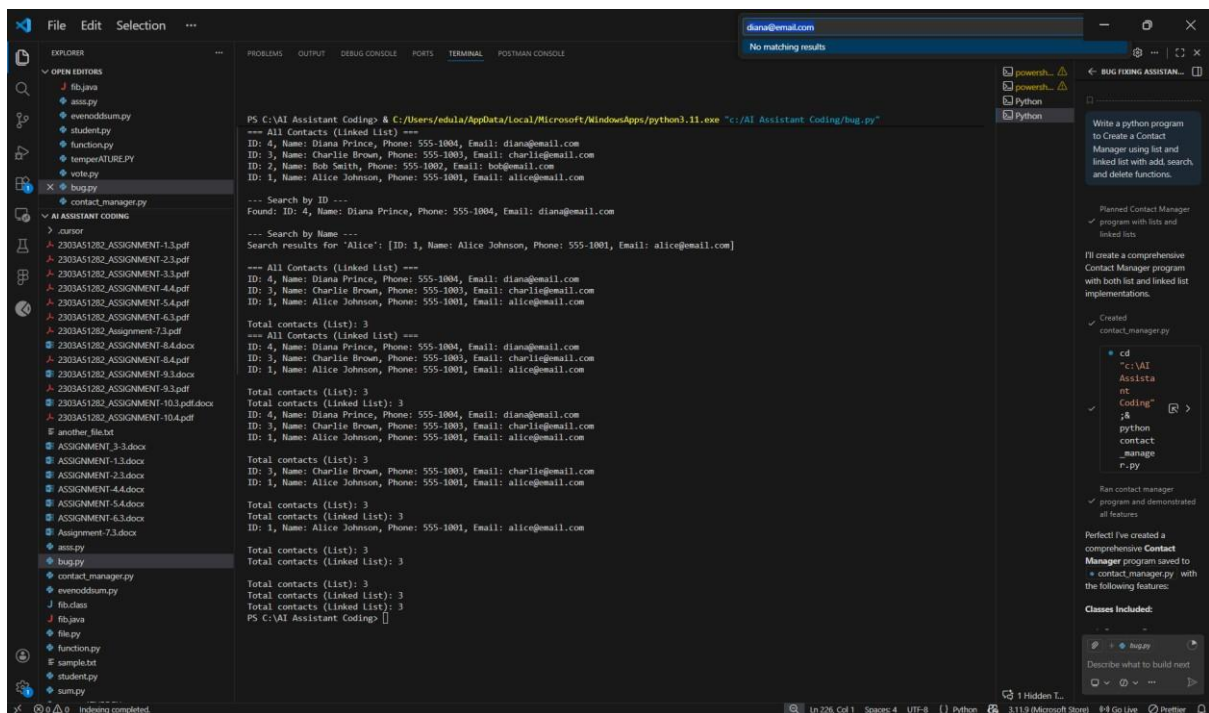
    # Delete a contact
    name = input("Enter name to delete: ")
    if manager.delete_contact(name):
        print(f"Contact '{name}' deleted successfully!")
    else:
        print("Contact not found!")

    # Update a contact
    name = input("Enter name to update: ")
    phone = input("Enter new phone: ")
    email = input("Enter new email: ")
    if manager.update_contact(name, phone, email):
        print(f"Contact '{name}' updated successfully!")
    else:
        print("Contact not found!")

    # Display all contacts
    manager.display_contacts()

    print("Total contacts: ", len(manager.contacts))
```

Output:



## Explanation:

- In an array, adding at the end is fast, but inserting in the middle is slow because elements must shift.
- In a linked list, insertion is fast because no shifting is needed.
- Searching takes the same time in both (you must check each element).
- Deleting in an array is slower due to shifting elements.
- Linked list is better for frequent insertions and deletions.

## Task 2:

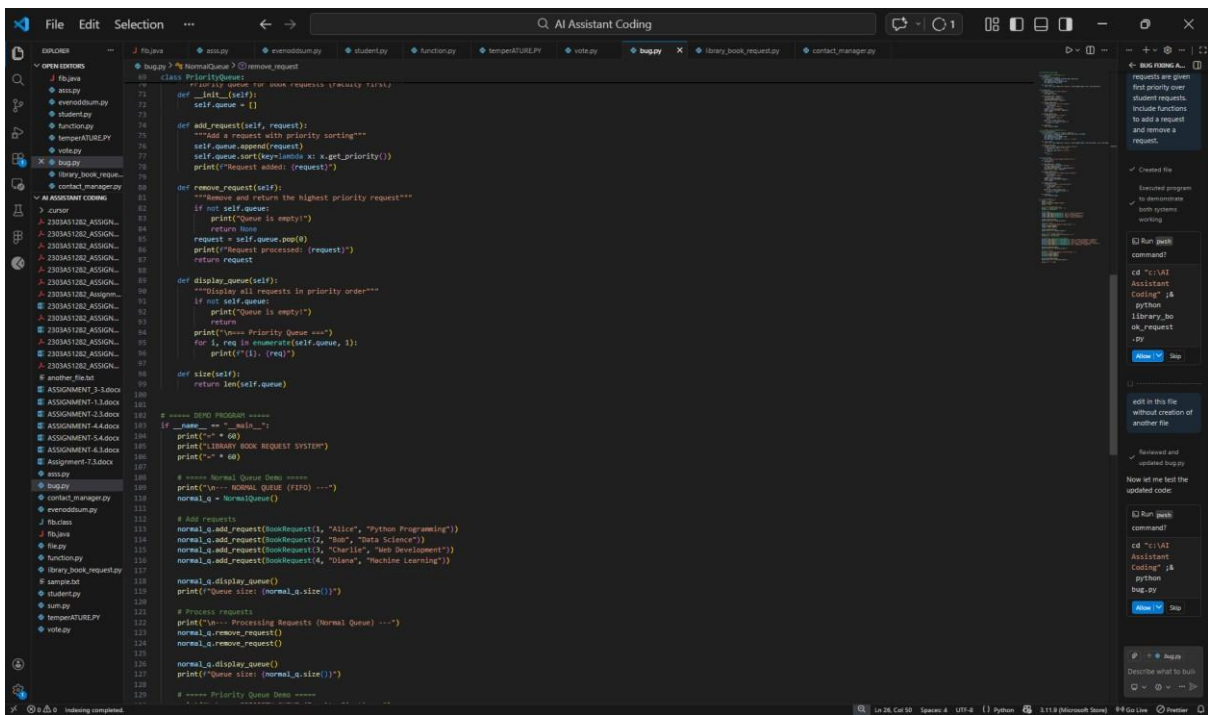
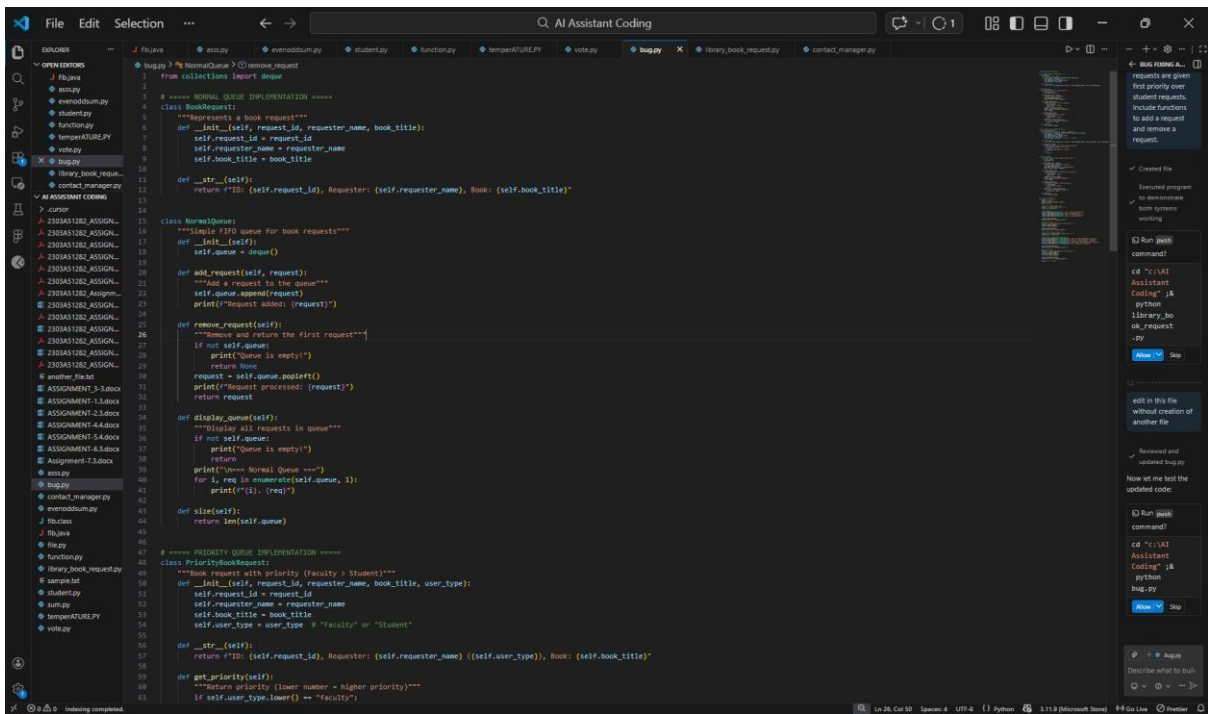
### Library Book Search System (Queues & Priority Queues) Scenario

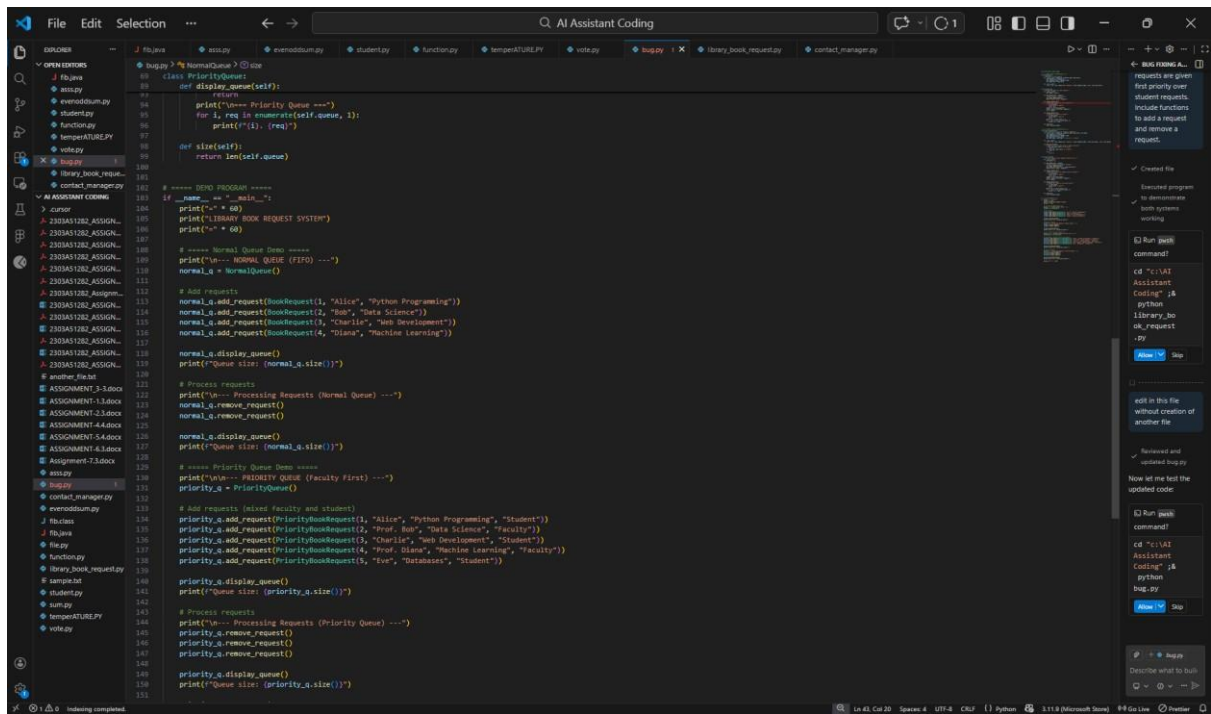
The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

## Prompt:

Write a Python program for a library book request system. First, make a normal queue where requests are handled in the order they come. Then, make another version where faculty requests are given first priority over student requests. Include functions to add a request and remove a request.

## Code:





## Output:

```
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"

=== Priority Queue ===
1. ID: 3, Requester: Charlie (Student), Book: Web Development
2. ID: 5, Requester: Eve (Student), Book: Databases
Queue size: 2

=====
PS C:\AI Assistant Coding>
```

## Explanation:

- Queue (FIFO) → First request comes, first served.(If a student requests first, they get the book first.)
- Priority Queue → Faculty requests are served before students, even if they come later.
- enqueue() → Adds a request to the system.
- dequeue() → Removes and processes the next request.

## Task 3: Emergency Help Desk (Stack Implementation)

### Scenario

SR University's IT Help Desk receives technical support tickets from students and staff.

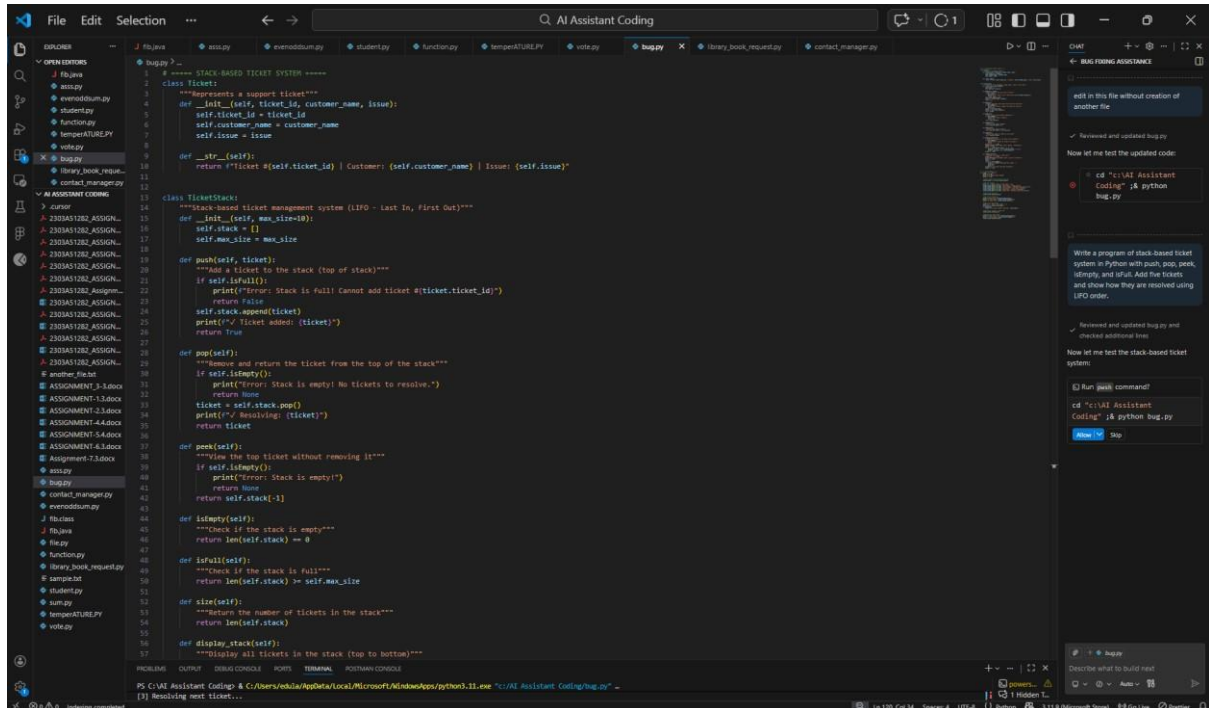
While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.



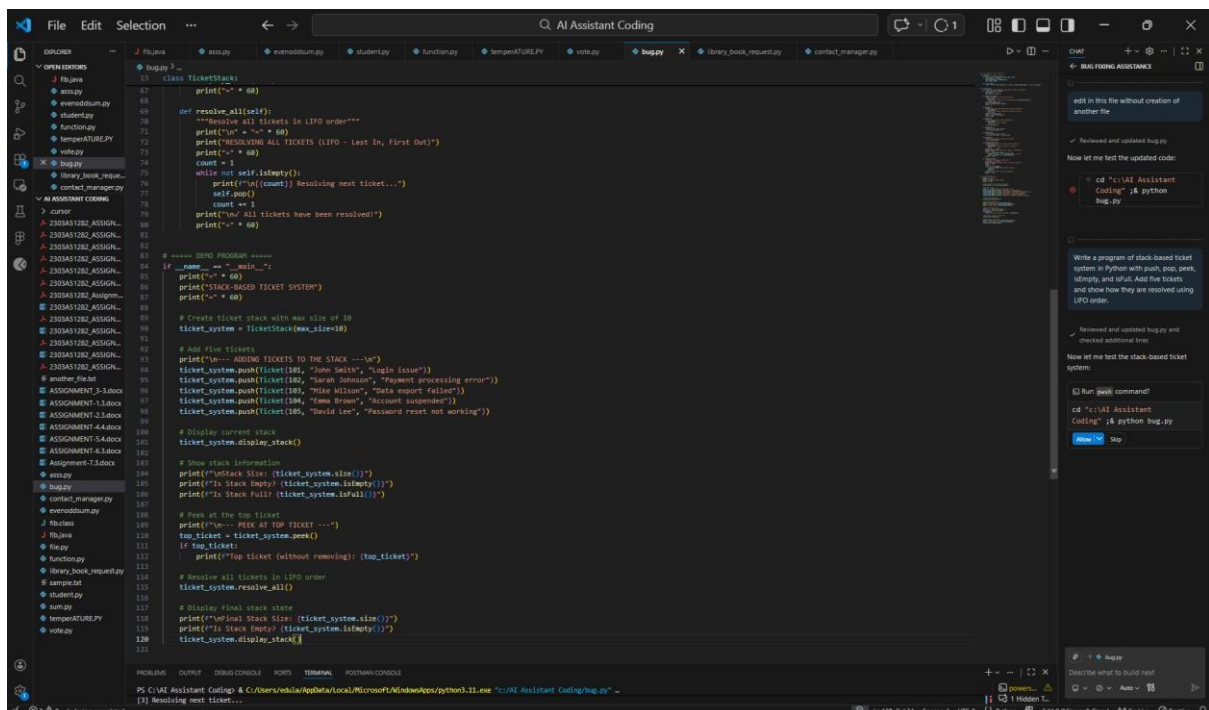
## Prompt:

Write a program of stack-based ticket system in Python with push, pop, peek, isEmpty, and isFull. Add five tickets and show how they are resolved using LIFO order.

## Code:

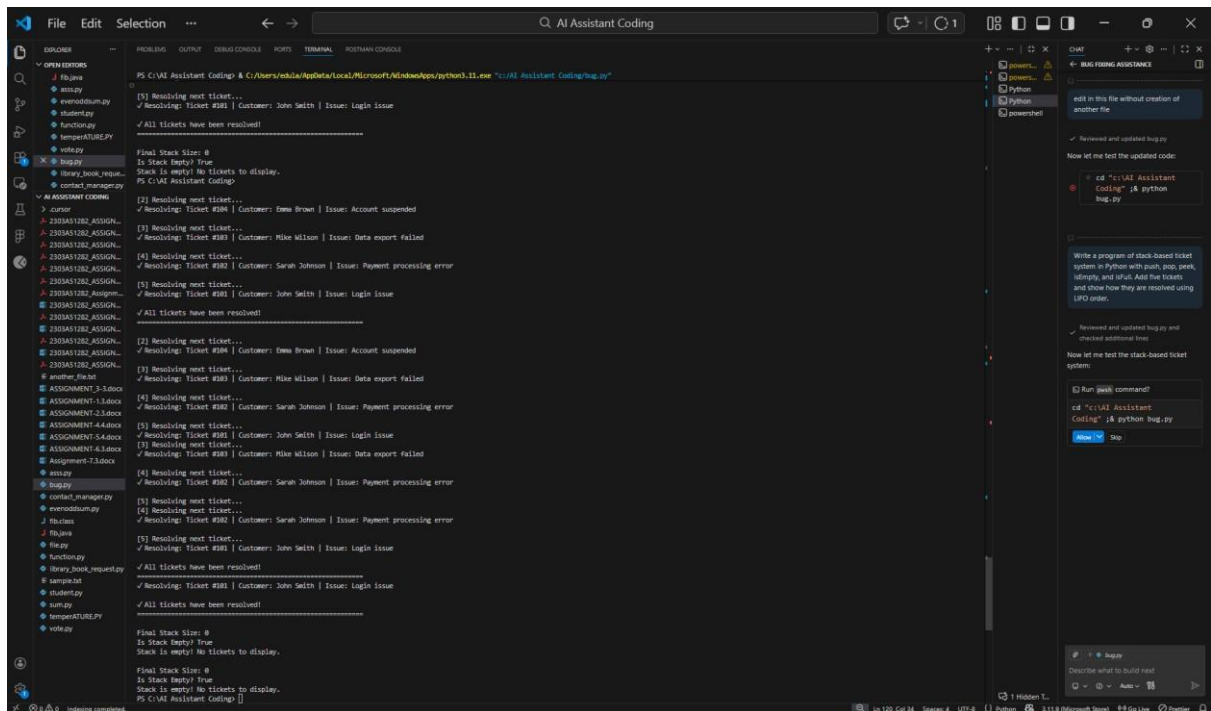


```
1 # ===== STACK-BASED TICKET SYSTEM =====
2 class Ticket:
3     """Represents a support ticket"""
4     def __init__(self, ticket_id, customer_name, issue):
5         self.ticket_id = ticket_id
6         self.customer_name = customer_name
7         self.issue = issue
8
9     def __str__(self):
10         return f"Ticket #{self.ticket_id} | Customer: {self.customer_name} | Issue: {self.issue}"
11
12 class TicketStack:
13     """Stack-based ticket management system (LIFO - Last In, First Out)"""
14     def __init__(self, max_size=10):
15         self.stack = []
16         self.max_size = max_size
17
18     def push(self, ticket):
19         """Add a ticket to the stack (top of stack)"""
20         if self.isFull():
21             print(f"Error: Stack is full! Cannot add ticket #{ticket.ticket_id}")
22             return False
23         self.stack.append(ticket)
24         print(f"/> Resolving ticket: {ticket}")
25         return True
26
27     def pop(self):
28         """Remove and return the ticket from the top of the stack"""
29         if self.isEmpty():
30             print(f"Error: Stack is empty! No tickets to resolve.")
31             return None
32         ticket = self.stack.pop()
33         print(f"/> Resolving ticket: {ticket}")
34         return ticket
35
36     def peek(self):
37         """View the top ticket without removing it"""
38         if self.isEmpty():
39             print(f"Error: Stack is empty!")
40             return None
41         return self.stack[-1]
42
43     def isEmpty(self):
44         """Check if the stack is empty"""
45         return len(self.stack) == 0
46
47     def isFull(self):
48         """Check if the stack is full"""
49         return len(self.stack) == self.max_size
50
51     def size(self):
52         """Return the number of tickets in the stack"""
53         return len(self.stack)
54
55     def display_stack(self):
56         """Display all tickets in the stack (top to bottom)"""
```



```
57     print(f"<--> {00}")
58
59     def resolve_all(self):
60         """Resolve all tickets in LIFO order"""
61         print(f"<--> {00}")
62         print(f"RESOLVING ALL TICKETS (LIFO - Last In, First Out)")
63         print(f"<--> {00}")
64         count = 1
65         while not self.isEmpty():
66             print(f"/> Resolving ticket: {ticket}")
67             self.pop()
68             count += 1
69         print(f"/> All tickets have been resolved!")
70         print(f"<--> {00}")
71
72 # ===== DEMO PROGRAM =====
73 if __name__ == "__main__":
74     print(f"<--> {00}")
75     print(f"STACK-BASED TICKET SYSTEM")
76     print(f"<--> {00}")
77
78     # Create ticket stack with max size of 10
79     ticket_system = TicketStack(max_size=10)
80
81     # Add five tickets
82     print(f"<--> ADDING TICKETS TO THE STACK --->")
83     ticket_system.push(ticket(101, "John Smith", "Login issue"))
84     ticket_system.push(ticket(102, "Sarah Johnson", "Payment processing error"))
85     ticket_system.push(ticket(103, "Mike Wilson", "Data export failure"))
86     ticket_system.push(ticket(104, "Emma Brown", "Account suspended"))
87     ticket_system.push(ticket(105, "David Lee", "Password reset not working"))
88
89     # Display current stack
90     ticket_system.display_stack()
91
92     # Show stack information
93     print(f"/> Stack Size: {ticket_system.size()}")
94     print(f"/> Is Stack Empty? {ticket_system.isEmpty()}")
95     print(f"/> Is Stack Full? {ticket_system.isFull()}")
96
97     # Peek at the top ticket
98     print(f"<--> PEAK AT TOP TICKET --->")
99     top_ticket = ticket_system.peek()
100     if top_ticket:
101         print(f"/> Top ticket (without removing): {top_ticket}")
102
103     # Resolve all tickets in LIFO order
104     ticket_system.resolve_all()
105
106     # Display final stack state
107     print(f"/> Final Stack Size: {ticket_system.size()}")
108     print(f"/> Is Stack Empty? {ticket_system.isEmpty()}")
109     ticket_system.display_stack()
```

## Output:



## Explanation:

The program uses a stack to manage help desk tickets.

A stack works in last in, first solved order.

When a new ticket is raised, it is added to the top.

When solving a ticket, the most recent one is handled first.

The program can also check if there are no tickets left or if the stack is full.

## Task 4:

Hash Table

### Objective

To implement a Hash Table and understand collision handling.

### Prompt:

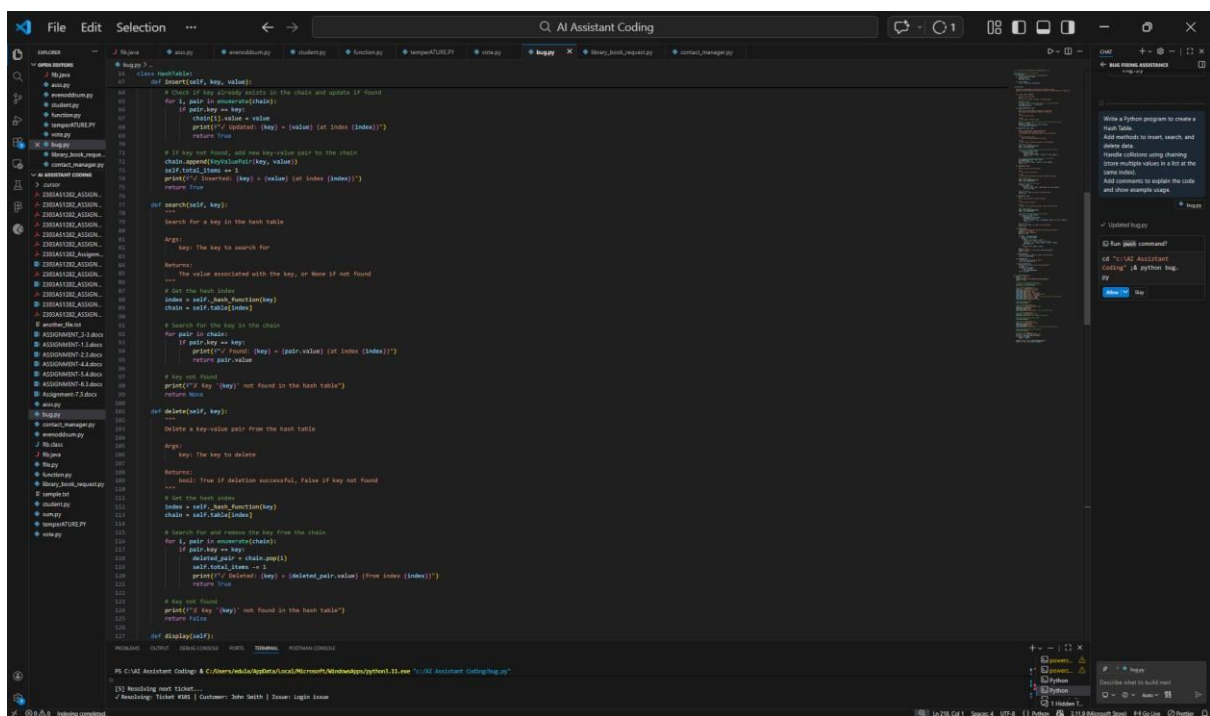
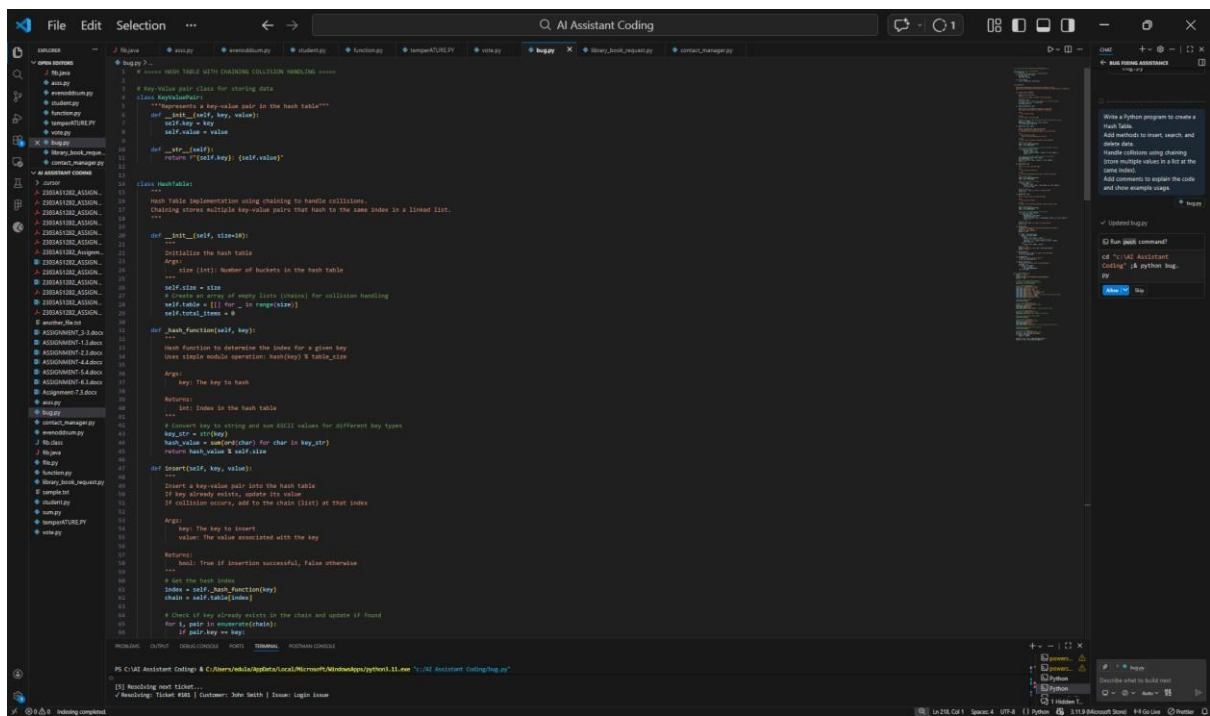
Write a Python program to create a Hash Table.

Add methods to insert, search, and delete data.

Handle collisions using chaining (store multiple values in a list at the same index).

Add comments to explain the code and show example usage.

### Code:







## Task 5:

### Real-Time Application Challenge Scenario

Design a Campus Resource Management System with the following features:

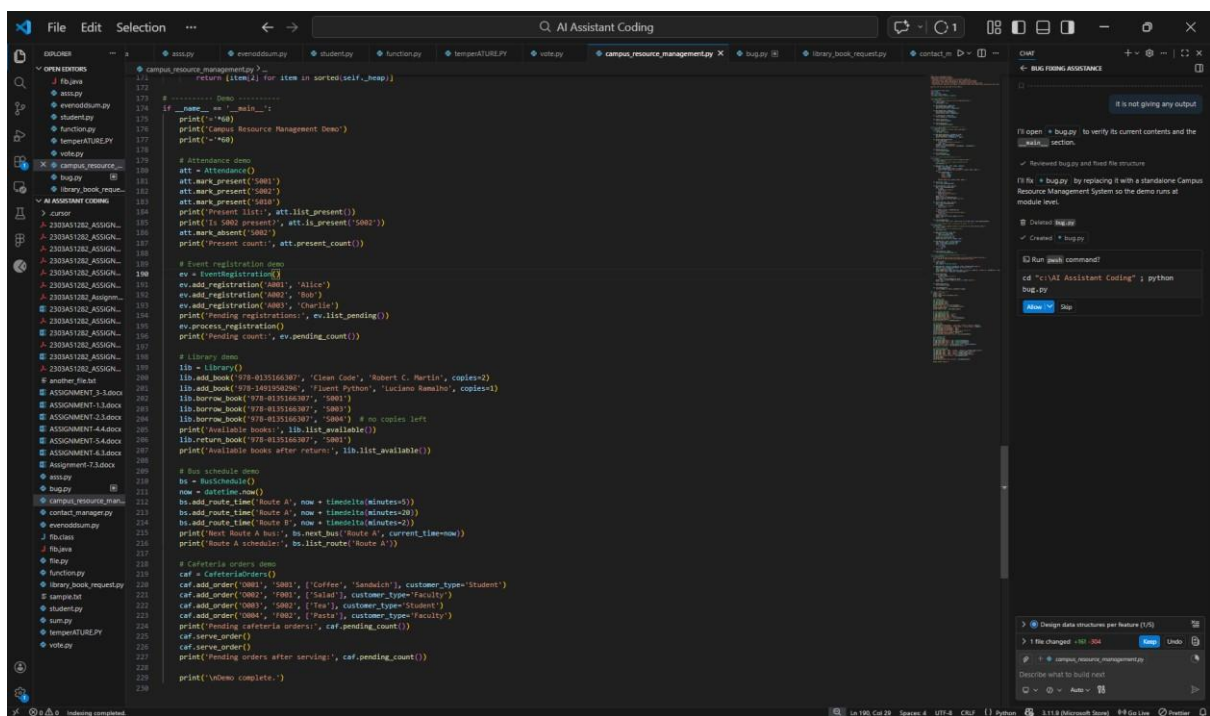
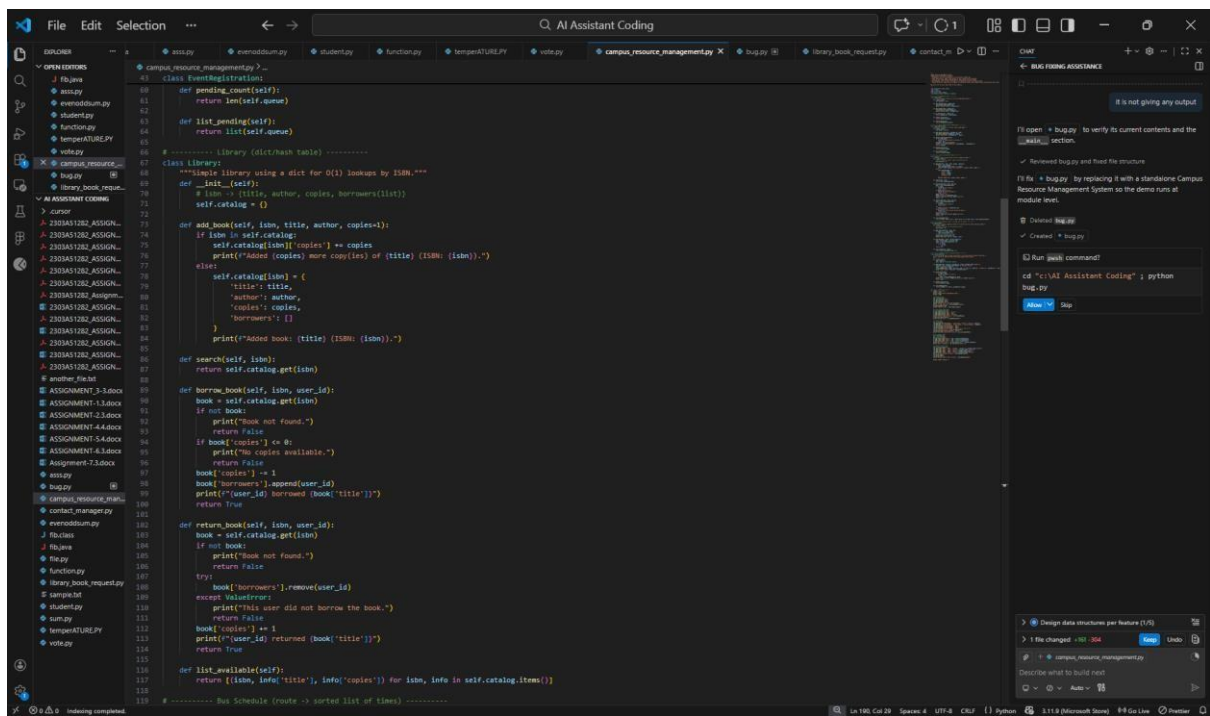
- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

## Prompt:

Create a Campus Resource Management System in Python. For each feature (Attendance, Event Registration, Library, Bus Schedule, Cafeteria Orders), choose the best data structure

## Code:

```
1 # campus_resource_management.py
2 """
3 Campus Resource Management System
4 Features and chosen data structures:
5 - Attendance: set (O(1) add/check) to track present student IDs
6 - Event Registration: deque (FIFO queue) for fair registration processing
7 - Library: dict (O(1) lookup) mapping book ID to book record for fast lookup
8 - Bus Schedule: dict of route -> sorted list of departure times (list kept sorted)
9 - Cafeteria Orders: heap (priority queue) to prioritize faculty over students while preserving arrival order
10 """
11
12 Run this file to see a small demo of each feature.
13 """
14
15 from collections import deque
16 import heapq
17 import sys
18 from bisect import bisect
19 from datetime import datetime, timedelta
20
21 # ===== Attendance (set) =====
22 class Attendance:
23     """Track attendance using a set for O(1) add/remove/check."""
24     def __init__(self):
25         self.present = set()
26
27     def mark_present(self, student_id):
28         self.present.add(student_id)
29         print(f"Marked present: {student_id}")
30
31     def mark_absent(self, student_id):
32         self.present.discard(student_id)
33         print(f"Marked absent: {student_id}")
34
35     def is_present(self, student_id):
36         return student_id in self.present
37
38     def present_count(self):
39         return len(self.present)
40
41     def list_present(self):
42         return sorted(self.present)
43
44 # ===== Event Registration (FIFO queue) =====
45 class EventRegistration:
46     """Register attendees in arrival order using deque."""
47     def __init__(self):
48         self.queue = deque()
49
50     def add_registration(self, attendee_id, name):
51         self.queue.append((attendee_id, name))
52         print(f"Registered: {attendee_id} - {name}")
53
54     def process_registration(self):
55         if not self.queue:
56             print("No registrations to process.")
57             return None
58         attendee = self.queue.popleft()
59         print(f"Processing registration: {attendee[0]} - {attendee[1]}")
60         return attendee
61
62     def pending_count(self):
63         return len(self.queue)
```



Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL POSTMAN CONSOLE

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
Is empty: False
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/campus_resource_management.py"

=====
Campus Resource Management Demo
=====

Marked present: 5001
Marked present: 5002
Marked present: 5010
Present list: ['5001', '5002', '5010']
Is 5002 present? True
Marked absent: 5002
Present count: 2
Registered: A001 - Alice
Registered: A002 - Bob
Registered: A003 - Charlie
Pending registrations: [('A001', 'Alice'), ('A002', 'Bob'), ('A003', 'Charlie')]
Processed registration: A001 - Alice
Pending count: 2
Added book: Clean Code (ISBN: 978-0135166307).
Added book: Fluent Python (ISBN: 978-1491958296).
5001 borrowed Clean Code
5003 borrowed Clean Code
No copies available.
Available books: [('978-0135166307', 'Clean Code', 0), ('978-1491958296', 'Fluent Python', 1)]
5001 returned Clean Code
Available books after return: [('978-0135166307', 'Clean Code', 1), ('978-1491958296', 'Fluent Python', 1)]
Added bus time for Route A: 2026-02-18 10:42:24.367227
Added bus time for Route A: 2026-02-18 10:57:24.367227
Added bus time for Route B: 2026-02-18 10:39:24.367227
Next Route A bus: 2026-02-18 10:42:24.367227
Route A schedule: [datetime.datetime(2026, 2, 18, 10, 42, 24, 367227), datetime.datetime(2026, 2, 18, 10, 57, 24, 367227)]
Order added: 0001 (Student)
Order added: 0002 (Faculty)
Order added: 0003 (Student)
Order added: 0004 (Faculty)
Pending cafeteria orders: 4
Serving order: 0002 (Faculty)
Serving order: 0004 (Faculty)
Pending orders after serving: 2

Demo complete.
PS C:\AI Assistant Coding>
```

## Explanation:

Library Book Borrowing using a queue:

- The queue stores student names who request a book.
- When a student requests a book, we use `enqueue()` to add them to the queue.
- When a book becomes available, we use `dequeue()` to give it to the first student in line.
- This ensures fairness because the first requester gets the book first.